

Lab 5.3. Use Lists in Kotlin

1. Before you begin

It's common to make lists for all sorts of situations in your everyday life such as a list of things to do, a list of guests for an event, a wish list, or a grocery list. In programming, lists are also very useful. For example, there could be a list of news articles, songs, calendar events, or social media posts within an app.

Learning how to create and use lists is an important programming concept to add to your toolbox, and it will enable you to create more sophisticated apps.

In this codelab, you will use the Kotlin Playground to become familiar with lists in Kotlin and create a program for ordering different variations of noodle soup. Are you hungry yet?

Prerequisites

- Familiar with using the [Kotlin Playground](#) for creating and editing Kotlin programs.
- Familiar with basic Kotlin programming concepts from [Unit 1](#) of the Android Basics in Kotlin course: the `main()` function, functions arguments and return values, variables, data types and operations, as well as control flow statements.
- Able to define a Kotlin class, create an object instance from it, and access its properties and methods.
- Able to create subclasses and understand how they inherit from each other.

What you'll learn

- How to create and use lists in Kotlin
- The difference between the `List` and `MutableList`, and when to use each one
- How to iterate over all items of a list and perform an action on each item.

What you'll build

- You will experiment with lists and list operations in the [Kotlin Playground](#).
- You will create a food ordering program that uses lists in the [Kotlin Playground](#).
- Your program will be able to create an order, add noodles and vegetables to it, and then calculate the total cost of the order.

What you need

- A computer with an internet connection to access the [Kotlin Playground](#).

2. Introduction to Lists

In earlier codelabs, you learned about basic data types in Kotlin such as `Int`, `Double`, `Boolean`, and `String`. They allow you to store a certain type of value within a variable. But what if you want to store more than one value? That is where having a `List` data type is useful.

A list is a collection of items with a specific order. There are two types of lists in Kotlin:

- Read-only list: `List` cannot be modified after you create it.
- Mutable list: `MutableList` can be modified after you create it, meaning you can add, remove, or update its elements.

When using `List` or `MutableList`, you must specify the type of element that it can contain. For example, `List<Int>` holds a list of integers and `List<String>` holds a list of Strings. If you define a `Car` class in your program, you can have a `List<Car>` that holds a list of `Car` object instances.

The best way to understand lists is to try them out.

Create a List

1. Open the [Kotlin Playground](#) and delete the existing code provided.
2. Add an empty `main()` function. All of the following code steps will go inside this `main()` function.

```
fun main() {  
  
}
```

3. Inside `main()`, create a variable called `numbers` of type `List<Int>` because this will contain a read-only list of integers. Create a new `List` using the Kotlin standard library function `listOf()`, and pass in the elements of the list as arguments separated by commas. `listOf(1, 2, 3, 4, 5, 6)` returns a read-only list of integers from 1 through 6.

```
val numbers: List<Int> = listOf(1, 2, 3, 4, 5, 6)
```

4. If the type of the variable can be guessed (or inferred) based on the value on the right hand side of the assignment operator (`=`), then you can omit the data type of the variable. Hence, you can shorten this line of code to the following:

```
val numbers = listOf(1, 2, 3, 4, 5, 6)
```

5. Use `println()` to print the `numbers` list.

```
println("List: $numbers")
```

Remember that putting `$` in the string means that what follows is an expression that will be evaluated and added to this string (see [string templates](#)). This line of code could also be written as `println("List: " + numbers)`.

6. Retrieve the size of a list using the `numbers.size` property, and print that out too.

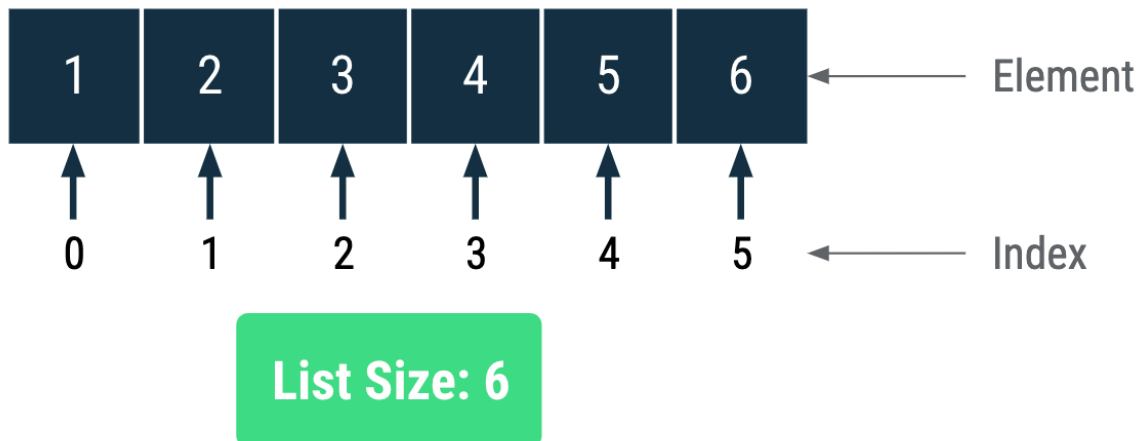
```
println("Size: ${numbers.size}")
```

7. Run your program. The output is a list of all elements of the list and the size of the list. Notice the brackets `[]`, indicating that this is a `List`. Inside the brackets are the elements of `numbers`, separated by commas. Also observe that the elements are in the same order as you created them.

```
List: [1, 2, 3, 4, 5, 6]  
Size: 6
```

Access list elements

The functionality specific to lists is that you can access each element of a list by its index, which is an integer number that represents the position. This is a diagram of the `numbers` list we created, showing each element and its corresponding index.



The index is actually an offset from the first element. For example, when you say `list[2]` you are not asking for the second element of the list, but for the element that is 2 positions offset from the first element. Hence `list[0]` is the first element (zero offset), `list[1]` is the second element (offset of 1), `list[2]` is the third element (offset of 2), and so on.

Add the following code after the existing code in the `main()` function. Run the code after each step, so you can verify the output is what you expect.

1. Print the first element of the list at index 0. You could call the `get()` function with the desired index as `numbers.get(0)` or you can use shorthand syntax with square brackets around the index as `numbers[0]`.

```
println("First element: ${numbers[0]}")
```

2. Next print the second element of the list at index 1.

```
println("Second element: ${numbers[1]}")
```

Valid index values ("indices") of a list go from 0 to the last index, which is the size of the list minus 1. That means for your `numbers` list, the indices run from 0 to 5.

3. Print the last element of the list, using `numbers.size - 1` to calculate its index, which should be 5. Accessing the element at the 5th index should return 6 as the output.

```
println("Last index: ${numbers.size - 1}")
println("Last element: ${numbers[numbers.size - 1]}")
```

4. Kotlin also supports `first()` and `last()` operations on a list. Try calling `numbers.first()` and `numbers.last()` and see the output.

```
println("First: ${numbers.first()}")
println("Last: ${numbers.last()}")
```

You'll notice that `numbers.first()` returns the first element of the list and `numbers.last()` returns the last element of the list.

5. Another useful list operation is the `contains()` method to find out if a given element is in the list. For example, if you have a list of employee names in a company, you can use the `contains()` method to find out if a given name is present in the list.

On your `numbers` list, call the `contains()` method with one of the integers that is present in the list. `numbers.contains(4)` would return the value `true`. Then call the `contains()` method with an integer that isn't in your list. `numbers.contains(7)` would return `false`.

```
println("Contains 4? ${numbers.contains(4)}")
println("Contains 7? ${numbers.contains(7)}")
```

6. Your completed code should look like this. The comments are optional.

```
fun main() {
    val numbers = listOf(1, 2, 3, 4, 5, 6)
    println("List: $numbers")
    println("Size: ${numbers.size}")

    // Access elements of the list
    println("First element: ${numbers[0]}")
    println("Second element: ${numbers[1]}")
}
```

```

println("Last index: ${numbers.size - 1}")
println("Last element: ${numbers[numbers.size - 1]}")
println("First: ${numbers.first()}")
println("Last: ${numbers.last()}")

// Use the contains() method
println("Contains 4? ${numbers.contains(4)}")
println("Contains 7? ${numbers.contains(7)}")
}

```

7. Run your code. This is the output.

```

List: [1, 2, 3, 4, 5, 6]
Size: 6
First element: 1
Second element: 2
Last index: 5
Last element: 6
First: 1
Last: 6
Contains 4? true
Contains 7? false

```

Lists are read-only

1. Delete the code within the Kotlin Playground and replace with the following code. The `colors` list is initialized to a list of 3 colors represented as `Strings`.

```

fun main() {
    val colors = listOf("green", "orange", "blue")
}

```

2. Remember that you cannot add or change elements in a read-only `List`. See what happens if you try to add an item to the list or try to modify an element of the list by setting it equal to a new value.

```

colors.add("purple")
colors[0] = "yellow"

```

3. Run your code and you get several error messages. In essence, the errors say that the `add()` method does not exist for `List`, and that you are not allowed to change the value of an element.

```

❶ colors.add("purple")
❷ colors[0] = "yellow"
}

```

- ❶ Unresolved reference: add
- ❷ Unresolved reference. None of the following candidates is applicable because of receiver type mismatch: public inline operator fun kotlin.text.StringBuilder /* = java.lang.StringBuilder */.set(index: Int, value: Char): Unit defined in kotlin.text
- ❸ No set method providing array access

4. Remove the incorrect code.

You've seen firsthand that it's not possible to change a read-only list. However, there are a number of operations on lists that don't change the list, but will return a new list. Two of those are `reversed()` and `sorted()`. The `reversed()` function returns a new list where the elements are in the reverse order, and `sorted()` returns a new list where the elements are sorted in ascending order.

1. Add code to reverse the `colors` list. Print the output. This is a new list that contains the elements of `colors` in reverse order.
2. Add a second line of code to print the original `list`, so you can see that the original list has not changed.

```

println("Reversed list: ${colors.reversed()}")
println("List: $colors")

```

3. This is the output of the two lists printed.

```

Reversed list: [blue, orange, green]
List: [green, orange, blue]

```

4. Add code to return a sorted version of a `List` using the [sorted\(\)](#) function.

```

println("Sorted list: ${colors.sorted()}")

```

The output is a new list of colors sorted in alphabetical order. Cool!

```

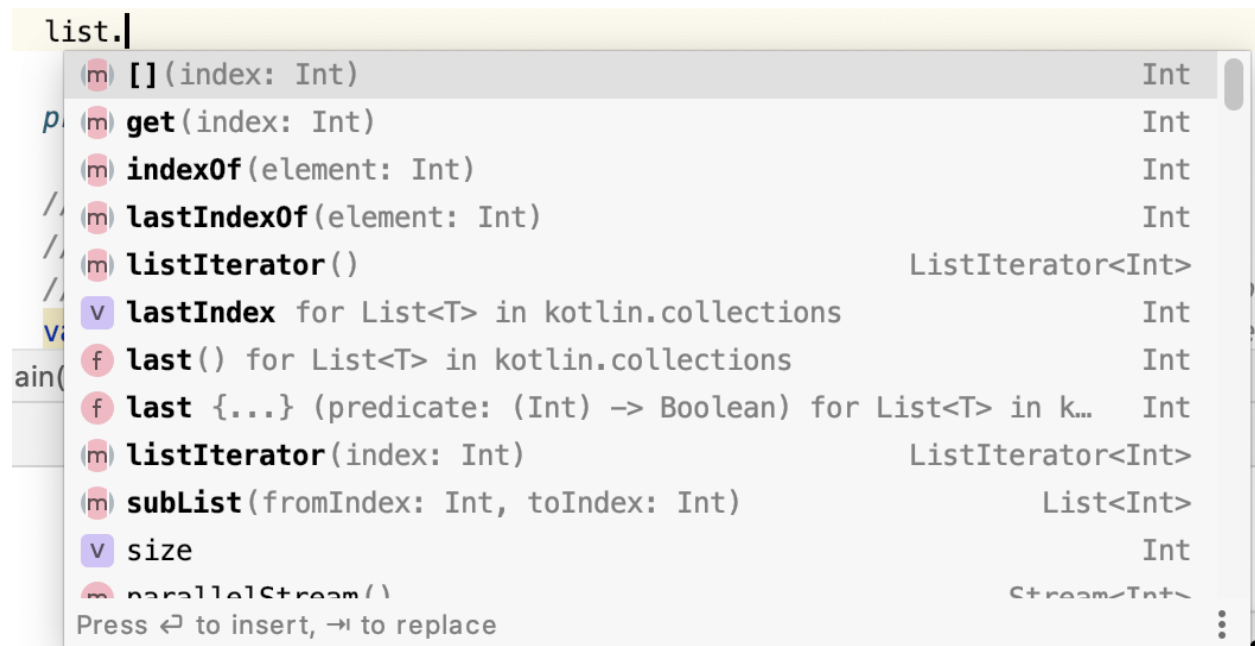
Sorted list: [blue, green, orange]

```

5. You can also try the `sorted()` function on a list of unsorted numbers.

```
val oddNumbers = listOf(5, 3, 7, 1)
println("List: $oddNumbers")
println("Sorted list: ${oddNumbers.sorted()}")
List: [5, 3, 7, 1]
Sorted list: [1, 3, 5, 7]
```

Note: Don't worry about having to remember all of the possible list operations. When you are developing apps in Android Studio, as you work with lists and other data types, Android Studio will show you the functions and properties available for them, as for example in the following screenshot.



By now, you can see the usefulness of being able to create lists. However it would be nice to be able to modify the list after creation, so let's look at mutable lists next.

[3. Introduction to Mutable Lists](#)

Mutable lists are lists that can be modified after creation. You can add, remove, or change items. You can do everything you can do with read-only lists as well. Mutable lists are of type [MutableList](#), and you can create them by calling [mutableListOf\(\)](#).

Create a MutableList

1. Delete the existing code in `main()`.
2. Within the `main()` function, create an empty mutable list and assign it to a `val` variable called `entrees`.

```
val entrees = mutableListOf()
```

This results in the following error, if you try to run your code.

```
Not enough information to infer type variable T
```

As mentioned earlier, when you create a `MutableList` or `List`, Kotlin tries to infer what type of elements the list contains from the arguments passed. For example, if you write `listOf("noodles")`, Kotlin infers that you want to create a list of `String`. When you initialize an empty list without elements, Kotlin cannot infer the type of the elements, so you have to explicitly state the type. Do this by adding the type in angle brackets right after `mutableListOf` or `listOf`. (In [documentation](#), you may see this as `<T>` where `T` stands for type parameter).

3. Correct the variable declaration to specify that you want to create a mutable list of type `String`.

```
val entrees = mutableListOf<String>()
```

Another way you could have fixed the error is by specifying the data type of the variable upfront.

```
val entrees: MutableList<String> = mutableListOf()
```

Note: You can use `val` for a mutable list because the `entrees` variable contains a reference to the list, and that reference doesn't change even if the contents of the list do.

4. Print the list.

```
println("Entrees: $entrees")
```

5. The output shows `[]` for an empty list.

```
Entrees: []
```

Add elements to a list

Mutable lists become interesting when you add, remove, and update elements.

1. Add "noodles" to the list with `entrees.add("noodles")`. The [add\(\)](#) function returns `true` if adding the element to the list succeeded, `false` otherwise.
2. Print the list to confirm that "noodles" has actually been added.

```
println("Add noodles: ${entrees.add("noodles")}")
println("Entrees: $entrees")
```

The output is:

```
Add noodles: true
Entrees: [noodles]
```

3. Add another item "spaghetti" to the list.

```
println("Add spaghetti: ${entrees.add("spaghetti")}")
println("Entrees: $entrees")
```

The resulting `entrees` list now contains two items.

```
Add spaghetti: true
Entrees: [noodles, spaghetti]
```

Instead of adding elements one by one using `add()`, you can add multiple elements at a time using [addAll\(\)](#) and pass in a list.

4. Create a list of `moreItems`. You won't have to change it, so make it a `val` and immutable.

```
val moreItems = listOf("ravioli", "lasagna", "fettuccine")
```

5. Using `addAll()`, add all the items from the new list to `entrees`. Print the resulting list.

```
println("Add list: ${entrees.addAll(moreItems)}")
println("Entrees: $entrees")
```

The output shows that adding the list was successful. The `entrees` list now has a total of 5 items.

```
Add list: true
Entrees: [noodles, spaghetti, ravioli, lasagna, fettuccine]
```

6. Now try adding a number to this list.

```
entrees.add(10)
```

This fails with an error:

```
The integer literal does not conform to the expected type String
```

This is because the `entrees` list expects elements of type `String`, and you are trying to add an `Int`. Remember to only add elements of the correct data type to a list. Otherwise you will get a compile error. This is one way that Kotlin ensures your code is safer with type safety.

7. Remove the incorrect line of code, so your code compiles.

Remove elements from a list

1. Call [remove\(\)](#) to remove "spaghetti" from the list. Print the list again.

```
println("Remove spaghetti: ${entrees.remove("spaghetti")}")
println("Entrees: $entrees")
```

2. Removing "spaghetti" returns true because the element was present in the list and could be successfully removed. The list now only has 4 items left.

```
Remove spaghetti: true
Entrees: [noodles, ravioli, lasagna, fettuccine]
```

3. What happens if you try to remove an item that doesn't exist in the list? Try to remove "rice" from the list with `entrees.remove("rice")`.

```
println("Remove item that doesn't exist: ${entrees.remove("rice")}")
println("Entrees: $entrees")
```

The `remove()` method returns `false`, because the element does not exist and therefore could not be removed. The list remains unchanged with only 4 items still. Output:

```
Remove item that doesn't exist: false
Entrees: [noodles, ravioli, lasagna, fettuccine]
```

4. You can also specify the index of the element to remove. Use [removeAt\(\)](#) to remove the item at index 0.

```
println("Remove first element: ${entrees.removeAt(0)}")
println("Entrees: $entrees")
```

The return value of the `removeAt(0)` is the first element ("noodles") which got removed from the list. The `entrees` list now has 3 remaining items.

```
Remove first element: noodles
Entrees: [ravioli, lasagna, fettuccine]
```

5. If you want to clear the whole list, you can call [clear\(\)](#).

```
entrees.clear()
println("Entrees: $entrees")
```

The output shows an empty list now.

```
Entrees: []
```

6. Kotlin gives you a way to check if a list is empty using [isEmpty\(\)](#) function. Try printing out `entrees.isEmpty()`.

```
println("Empty? ${entrees.isEmpty()}")
```

The output should be true because the list is currently empty with 0 elements.

```
Empty? true
```

The `isEmpty()` method is useful if you want to do an operation on a list or access a certain element, but you want to make sure that the list is not empty first.

Here is all the code you wrote for mutable lists. The comments are optional.

```
fun main() {
    val entrees = mutableListOf<String>()
    println("Entrees: $entrees")

    // Add individual items using add()
    println("Add noodles: ${entrees.add("noodles")}")
    println("Entrees: $entrees")
    println("Add spaghetti: ${entrees.add("spaghetti")}")
    println("Entrees: $entrees")

    // Add a list of items using addAll()
    val moreItems = listOf("ravioli", "lasagna", "fettuccine")
    println("Add list: ${entrees.addAll(moreItems)}")
    println("Entrees: $entrees")

    // Remove an item using remove()
    println("Remove spaghetti: ${entrees.remove("spaghetti")}")
    println("Entrees: $entrees")
    println("Remove item that doesn't exist: ${entrees.remove("rice")}")
    println("Entrees: $entrees")

    // Remove an item using removeAt() with an index
    println("Remove first element: ${entrees.removeAt(0)}")
    println("Entrees: $entrees")

    // Clear out the list
    entrees.clear()
    println("Entrees: $entrees")

    // Check if the list is empty
    println("Empty? ${entrees.isEmpty()}")
}
```

4. Loop Through a List

To perform an operation on each item in a list, you can loop through the list (also known as iterating through the list). Loops can be used with `Lists` and `MutableLists`.

While loops

One type of loop is a [while](#) loop. A `while` loop starts with the `while` keyword in Kotlin. It contains a block of code (within curly braces) that gets executed over and over again, as long as the expression in the parentheses is true. To prevent the code from executing forever (called an infinite loop), the code block must contain logic that changes the value of the expression, so that eventually the expression will be false and you stop executing the loop. At that point, you exit the `while` loop, and continue with executing the code that comes after the loop.

```
while (expression) {  
    // While the expression is true, execute this code block  
}
```

Note: A `while` loop doesn't need to involve a list (example [here](#)) but it is useful for lists.

Use a `while` loop to iterate through a list. Create a variable to keep track of what `index` you're currently looking at in the list. This `index` variable will keep incrementing by 1 each time until you reach the last index of the list, after which you exit the loop.

1. Delete the existing code in the Kotlin Playground and have an empty `main()` function.
2. Let's say you are organizing a party. Create a list where each element represents the number of guests that RSVP'd from each family. First family said 2 people would come from their family. Second family said 4 of them would come, and so on.

```
val guestsPerFamily = listOf(2, 4, 1, 3)
```

3. Figure out how many total guests there will be. Write a loop to find the answer. Create a `var` for the total number of guests and initialize it to 0.

```
var totalGuests = 0
```

4. Initialize a `var` for the `index` variable, as described earlier.

```
var index = 0
```

5. Write a `while` loop to iterate through the list. The condition is to keep executing the code block, as long as the `index` value is less than the size of the list.

```
while (index < guestsPerFamily.size) {  
  
}
```

6. Within the loop, get the element of the list at the current `index` and add it to the total number of guests variable. Remember that `totalGuests += guestsPerFamily[index]` is the same as `totalGuests = totalGuests + guestsPerFamily[index]`.

Notice that the last line of the loop increments the `index` variable by 1 using `index++`, so that the next iteration of the loop will look at the next family in the list.

```
while (index < guestsPerFamily.size) {  
    totalGuests += guestsPerFamily[index]  
    index++  
}
```

7. After the `while` loop, you can print out the result.

```
while ... {  
    ...  
}
```

```
}  
println("Total Guest Count: $totalGuests")
```

8. Run the program and the output is as follows. You can verify this is the correct answer by manually adding up the numbers in the list.

Total Guest Count: 10

Here's the full code snippet:

```
val guestsPerFamily = listOf(2, 4, 1, 3)  
var totalGuests = 0  
var index = 0  
while (index < guestsPerFamily.size) {  
    totalGuests += guestsPerFamily[index]  
    index++  
}  
println("Total Guest Count: $totalGuests")
```

Note: Because doing a simple operation on a variable and storing it back in the variable is common, there is a shorthand for it using the `+=` operator. There are equivalent operators for minus (`-=`), multiplication (`*=`) and division (`/=`). See more details [here](#).

With a `while` loop, you had to write code to create a variable to keep track of the index, to get the element at the index in the list, and to update that index variable. There's an even faster and more concise way to iterate through a list. Use `for` loops!

For loops

A `for` loop is another type of loop. It makes looping through a list much easier. It starts with the `for` keyword in Kotlin with the code block in curly braces. The condition for executing the block of code is stated in parentheses.

```
for (number in numberList) {  
    // For each element in the list, execute this code block  
}
```

In this example, the variable `number` is set equal to the first element of `numberList` and the code block is executed. Then the `number` variable is automatically updated to be the next element of `numberList`, and the code block is executed again. This repeats for each element of the list, until the end of the `numberList` is reached.

1. Delete the existing code in the Kotlin Playground and replace with the following code:

```
fun main() {  
    val names = listOf("Jessica", "Henry", "Alicia", "Jose")  
}
```

2. Add a `for` loop to print all items in the `names` list.

```
for (name in names) {  
    println(name)  
}
```

This is much easier than if you had to write this as a `while` loop!

3. The output is:

```
Jessica  
Henry  
Alicia  
Jose
```

A common operation on lists is to do something with each element of the list.

4. Modify the loop to also print out the number of characters in that person's name. Hint: you can use the [length](#) property of a `String` to find the number of characters in that `String`.

```
val names = listOf("Jessica", "Henry", "Alicia", "Jose")  
for (name in names) {  
    println("$name - Number of characters: ${name.length}")  
}
```

Output:

```
Jessica - Number of characters: 7  
Henry - Number of characters: 5  
Alicia - Number of characters: 6  
Jose - Number of characters: 4
```

The code in the loop did not change the original `List`. It only affected what was printed.

It's pretty neat how you can write the instructions for what should happen for 1 list item, and the code gets executed for every list item! Using a loop can save you from typing out a lot of repetitive code.

Note: Here are some other variations of what you can do with `for` loops, including using them with ranges with specific steps (instead of incrementing by 1 each time).

```
for (item in list) print(item) // Iterate over items in a list  
  
for (item in 'b'..'g') print(item) // Range of characters in an alphabet  
  
for (item in 1..5) print(item) // Range of numbers  
  
for (item in 5 downTo 1) print(item) // Going backward
```

```
for (item in 3..6 step 2) print(item) // Prints: 3 5
```

You can find more in the documentation listed at the end of this codelab.

Note: A loop is a control flow structure like the `if / else` and `when` expressions you learned earlier because it dictates a particular flow for how the code should be executed.

Now that you've experimented with creating and using both lists and mutable lists, and learned about loops, it's time to apply this knowledge in a sample use case!

5. Put it all together

When it comes to ordering food at a local restaurant, there's usually multiple items within a single order for a customer. Using lists is ideal for storing information about an order. You'll also draw on your knowledge of classes and inheritance to create a more robust and scalable Kotlin program, instead of putting all the code within the `main()` function.

For the next series of tasks, create a Kotlin program that allows for ordering different combinations of food.

First take a look at this example output of the final code. Can you brainstorm what kinds of classes you would need to create to help organize all this data?

```
Order #1
Noodles: $10
Total: $10

Order #2
Noodles: $10
Vegetables Chef's Choice: $5
Total: $15
```

From the output, you'll notice that:

- there's a list of orders
- each order has a number
- each order can contain a list of items such as noodles and vegetables
- each item has a price
- each order has a total price, which is the sum of the prices of the individual items

You could create a class to represent an `Order`, and a class to represent each food item like `Noodles` or `Vegetables`. You may further observe that `Noodles` and `Vegetables` have some similarities because they are both food items and each have a price. You could consider creating an `Item` class with shared properties that both `Noodle` class and `Vegetable` class could inherit from. That way you don't have to duplicate logic in both `Noodle` class and `Vegetable` class.

1. You'll be provided with the following starter code. Professional developers often have to read other people's code, for example, if they're joining a new project or adding onto a feature that someone else created. Being able to read and understand code is an important skill to have.

Take some time to look over this code and understand what is happening. Copy and paste this code into the Kotlin Playground and run it. Be sure to delete any existing code in the Kotlin Playground before pasting in this new code. Observe the output and see if that helps you understand the code better.

```
open class Item(val name: String, val price: Int)

class Noodles : Item("Noodles", 10)

class Vegetables : Item("Vegetables", 5)

fun main() {
    val noodles = Noodles()
    val vegetables = Vegetables()
    println(noodles)
    println(vegetables)
}
```

2. You should see output similar to this:

```
Noodles@5451c3a8
Vegetables@76ed5528
```

Here's a more detailed explanation of the code. First there is a class called `Item`, where the constructor takes in 2 parameters: a `name` for the item (as a `String`) and a `price` (as an integer). Both properties do not change after they're passed in, so they are marked as `val`. Since `Item` is a parent class, and subclasses extend from it, the class is marked with the `open` keyword.

The `Noodles` class constructor takes in no parameters, but extends from `Item` and calls the superclass constructor by passing in "Noodles" as the name and the price of 10. The `Vegetables` class is similar but calls the superclass constructor with "Vegetables" and a price of 5.

The `main()` function initializes new object instances of `Noodles` and `Vegetables` classes, and prints them to the output.

Override `toString()` method

When you print an object instance to the output, the object's `toString()` method is called. In Kotlin, every class automatically inherits the `toString()` method. The default implementation of this method just returns the object type with a memory address for the instance. You should override `toString()` to return something more meaningful and user-friendly than

`Noodles@5451c3a8` and `Vegetables@76ed5528`.

1. Inside the `Noodles` class, override the `toString()` method and have it return the `name`. Remember that `Noodles` inherits the `name` property from its parent class `Item`.

```
class Noodles : Item("Noodles", 10) {  
    override fun toString(): String {  
        return name  
    }  
}
```

2. Repeat the same for the `Vegetables` class.

```
class Vegetables() : Item("Vegetables", 5) {  
    override fun toString(): String {  
        return name  
    }  
}
```

3. Run your code. The output looks better now:

```
Noodles  
Vegetables
```

In the next step, you will change the `Vegetables` class constructor to take in some parameters, and update the `toString()` method to reflect that additional information.

Customize vegetables in an order

To make the noodle soups more interesting, you can include different vegetables in your orders.

1. In the `main()` function, instead of initializing a `Vegetables` instance with no input arguments, pass in specific types of vegetables that the customer wants.

```
fun main() {  
    ...  
    val vegetables = Vegetables("Cabbage", "Sprouts", "Onion")  
    ...  
}
```

If you try to compile your code now, there will be an error that says:

```
Too many arguments for public constructor Vegetables() defined in Vegetables
```

You are now passing 3 `String` arguments into the `Vegetables` class constructor, so you will need to modify the `Vegetables` class.

2. Update the `Vegetables` class header to take in 3 string parameters, as shown in the following code:

```
class Vegetables(val topping1: String,
                 val topping2: String,
                 val topping3: String) : Item("Vegetables", 5) {
```

3. Now your code compiles again. However, this solution only works if your customers want to always order exactly three vegetables. If a customer wants to order one or five vegetables, they are out of luck.
4. Instead of using a property for each vegetable, you can fix the issue by accepting a list of vegetables (which can be any length) in the constructor for the `Vegetables` class. The `List` should contain only `Strings`, hence the type of the input parameter is `List<String>`.

```
class Vegetables(val toppings: List<String>) : Item("Vegetables", 5) {
```

This is not the most elegant solution because in `main()`, you would need to change your code to create a list of the toppings first before passing it into the `Vegetables` constructor.

```
Vegetables(listOf("Cabbage", "Sprouts", "Onion"))
```

There's an even better way to solve this.

5. In Kotlin, the `vararg` modifier allows you to pass a variable number of arguments of the same type into a function or constructor. In that way, you can supply the different vegetables as individual strings instead of a list.

Change the class definition of `Vegetables` to take a `vararg toppings` of type `String`.

```
class Vegetables(vararg val toppings: String) : Item("Vegetables", 5) {
```

Note: Only one parameter can be marked as `vararg` and is usually the last parameter in the list.

6. This code in the `main()` function will now work. You can create a `Vegetables` instance by passing in any number of topping `Strings`.

```
fun main() {
    ...
    val vegetables = Vegetables("Cabbage", "Sprouts", "Onion")
    ...
}
```

7. Now modify the `toString()` method of the `Vegetables` class so that it returns a `String` that also mentions the toppings in this format: `Vegetables Cabbage, Sprouts, Onion`.

Start with the name of the `Item` (`Vegetables`). Then use the `joinToString()` method to join all the toppings into a single string. Add these two parts together using the `+` operator with a space in between.

```
class Vegetables(vararg val toppings: String) : Item("Vegetables", 5) {
    override fun toString(): String {
        return name + " " + toppings.joinToString()
    }
}
```

8. Run your program and the output should be:

```
Noodles
Vegetables Cabbage, Sprouts, Onion
```

Note: To specify a different separator other than a comma, pass in the desired separator string as an argument to the `joinToString()` method. Example: `joinToString(" ")` to separate each item with a space.

9. When writing programs, you need to consider all possible inputs. When there's no input arguments to the `Vegetables` constructor, handle the `toString()` method in a more user-friendly way.

Since the customer does want vegetables, but didn't say which ones, one solution is to give them a default of the chef's choice vegetables.

Update the `toString()` method to return `Vegetables Chef's Choice` if there are no toppings passed in. Make use of the `isEmpty()` method that you learned about earlier.

```
override fun toString(): String {
    if (toppings.isEmpty()) {
        return "$name Chef's Choice"
    } else {
        return name + " " + toppings.joinToString()
    }
}
```

10. Update the `main()` function to test out both possibilities for creating a `Vegetables` instance without any constructor arguments and with several arguments.

```
fun main() {
    val noodles = Noodles()
    val vegetables = Vegetables("Cabbage", "Sprouts", "Onion")
    val vegetables2 = Vegetables()
    println(noodles)
    println(vegetables)
    println(vegetables2)
}
```

11. Verify the output is as expected.

```
Noodles
Vegetables Cabbage, Sprouts, Onion
Vegetables Chef's Choice
```

Create an order

Now that you have some food items, you can create an order. Encapsulate the logic for an order within an `Order` class in your program.

1. Think about what properties and methods would be reasonable for the `Order` class. If it helps, here is some sample output from the final code again.

```
Order #1
Noodles: $10
Total: $10
```

```
Order #2
Noodles: $10
Vegetables Chef's Choice: $5
Total: $15
```

```
Order #3
Noodles: $10
Vegetables Carrots, Beans, Celery: $5
Total: $15
```

```
Order #4
Noodles: $10
Vegetables Cabbage, Onion: $5
Total: $15
```

```
Order #5
Noodles: $10
Noodles: $10
Vegetables Spinach: $5
Total: $25
```

2. You may have come up with the following:

Order class

Properties: order number, list of items

Methods: add item, add multiple items, print order summary (including price)

3. Focusing on properties first, what should the data type of each property be? Should they be public or private to the class? Should they be passed in as arguments or defined within the class?
4. There's multiple ways to implement this, but here is one solution. Create a `class Order` that has an integer `orderNumber` constructor parameter.

```
class Order(val orderNumber: Int)
```

5. Since you may not know all the items in the order upfront, don't require the list of items to be passed in as an argument. Instead it can be declared as a top-level class variable, and initialize it as an empty `MutableList` that can hold elements of type `Item`. Mark the variable `private`, so that only this class can modify that list of items directly. This will protect the list from being modified in unexpected ways by code outside this class.

```
class Order(val orderNumber: Int) {  
    private val itemList = mutableListOf<Item>()  
}
```

6. Go ahead and add the methods to the class definition too. Feel free to pick reasonable names for each method, and you can leave the implementation logic within each method blank for now. Also decide on what function arguments and return values should be required.

```
class Order(val orderNumber: Int) {  
    private val itemList = mutableListOf<Item>()  
  
    fun addItem(newItem: Item) {  
    }  
  
    fun addAll(newItems: List<Item>) {  
    }  
  
    fun print() {  
    }  
}
```

7. The `addItem()` method seems the most straightforward, so implement that function first. It takes in a new `Item`, and the method should add it to the `itemList`.

```
fun addItem(newItem: Item) {  
    itemList.add(newItem)  
}
```

8. Implement the `addAll()` method next. It takes in a read-only list of items. Add all of those items to the internal list of items.

```
fun addAll(newItems: List<Item>) {  
    itemList.addAll(newItems)  
}
```

9. Then implement the `print()` method which prints a summary of all the items and their prices to the output, as well as the total price of the order.

First print out the order number. Then use a loop to iterate through all items in the order list. Print out each item and its respective price. Also keep a total of the price of the so far, and keep adding to it as you iterate through the list. Print out the total price at the end. Try to implement this logic yourself. If you need help, check the solution below.

You may want to include the currency symbol to make the output easier to read. Here's one way to implement the solution. This code uses the \$ currency symbol, but feel free to modify to your local currency symbol.

```
fun print() {
    println("Order #${orderNumber}")
    var total = 0
    for (item in itemList) {
        println("${item}: ${item.price}")
        total += item.price
    }
    println("Total: ${total}")
}
```

For each `item` in the `itemList`, print the `item` (which triggers `toString()` to be called on the `item`) followed by the `price` of the item. Also before the loop, initialize a `total` integer variable to be 0. Then keep adding to the total by adding the current item's price to the `total`.

Create Orders

1. Test out your code by creating `Order` instances within the `main()` function. Delete what you currently have in your `main()` function first.
2. You can use these sample orders or create your own. Experiment with different combinations of items within orders, making sure that you test out all code paths in your code. For example, test `addItem()` and `addAll()` methods within the `Order` class, create `Vegetables` instances with no arguments and with arguments, and so on.

```
fun main() {
    val order1 = Order(1)
    order1.addItem(Noodles())
    order1.print()

    println()

    val order2 = Order(2)
    order2.addItem(Noodles())
    order2.addItem(Vegetables())
    order2.print()

    println()

    val order3 = Order(3)
    val items = listOf(Noodles(), Vegetables("Carrots", "Beans", "Celery"))
    order3.addAll(items)
    order3.print()
}
```

3. The output for the above code should be the following. Verify that the total price is being added up properly.

Order #1

```
Noodles: $10
Total: $10
```

```
Order #2
Noodles: $10
Vegetables Chef's Choice: $5
Total: $15
```

```
Order #3
Noodles: $10
Vegetables Carrots, Beans, Celery: $5
Total: $15
```

Great job! It's looking like food orders now!

6. Improve your code

Keep a list of orders

If you were building a program that would actually be used in a noodle shop, it would be reasonable to keep track of a list of all customer orders.

1. Create a list to store all the orders. Is it a read-only list or mutable list?
2. Add this code to the `main()` function. Initialize the list to be empty at first. Then as each order gets created, add the order to the list.

```
fun main() {
    val ordersList = mutableListOf<Order>()

    val order1 = Order(1)
    order1.addItem(Noodles())
    ordersList.add(order1)

    val order2 = Order(2)
    order2.addItem(Noodles())
    order2.addItem(Vegetables())
    ordersList.add(order2)

    val order3 = Order(3)
    val items = listOf(Noodles(), Vegetables("Carrots", "Beans", "Celery"))
    order3.addAll(items)
    ordersList.add(order3)
}
```

Since the orders get added over time, the list should be a `MutableList` of type `Order`. Then use the `add()` method on `MutableList` to add each order.

3. Once you have a list of orders, you can use a loop to print each one. Print a blank line between orders so the output is easier to read.

```

fun main() {
    val ordersList = mutableListOf<Order>()

    ...

    for (order in ordersList) {
        order.print()
        println()
    }
}

```

This removes duplicate code in our `main()` function and makes the code easier to read! The output should be the same as before.

Implement Builder Pattern for Orders

To make your Kotlin code more concise, you can use the Builder pattern for creating orders. The Builder pattern is a design pattern in programming that allows you to build up a complex object in a step by step approach.

1. Instead of returning `Unit` (or nothing) from the `addItem()` and `addAll()` methods in `Order` class, return the changed `Order`. Kotlin provides the keyword `this` to reference the current object instance. Within the `addItem()` and `addAll()` methods, you return the current `Order` by returning `this`.

```

fun addItem(newItem: Item): Order {
    itemList.add(newItem)
    return this
}

fun addAll(newItems: List<Item>): Order {
    itemList.addAll(newItems)
    return this
}

```

2. In the `main()` function, you can now chain the calls together, as shown in the following code. This code creates a new `Order` and takes advantage of the Builder pattern.

```

val order4 = Order(4).addItem(Noodles()).addItem(Vegetables("Cabbage",
"Onion"))
ordersList.add(order4)

```

`Order(4)` returns an `Order` instance, which you can then call `addItem(Noodles())` on. The `addItem()` method returns the same `Order` instance (with the new state), and you can call `addItem()` again on it with vegetables. The returned `Order` result can be stored in the `order4` variable.

The existing code to create `Orders` still works, so that can be left unchanged. While it is not mandatory to chain these calls, it is a common and recommended practice that takes advantage of the function's return value.

3. At this point, you actually don't even need to store the order in a variable. In the `main()` function (before the final loop to print out the orders), create an `Order` directly and add it to the `orderList`. The code is also easier to read if each method call gets put on its own line.

```
ordersList.add(  
    Order(5)  
        .addItem(Noodles())  
        .addItem(Noodles())  
        .addItem(Vegetables("Spinach")))
```

4. Run your code, and this is the expected output:

```
Order #1  
Noodles: $10  
Total: $10  
  
Order #2  
Noodles: $10  
Vegetables Chef's Choice: $5  
Total: $15  
  
Order #3  
Noodles: $10  
Vegetables Carrots, Beans, Celery: $5  
Total: $15  
  
Order #4  
Noodles: $10  
Vegetables Cabbage, Onion: $5  
Total: $15  
  
Order #5  
Noodles: $10  
Noodles: $10  
Vegetables Spinach: $5  
Total: $25
```

Congratulations on finishing this codelab!

Now you've seen how useful it can be to store data in lists, to mutate lists, and loop through lists. Use this knowledge in the context of an Android app to display a list of data on screen in the next codelab!

[7. Solution code](#)

Here is the solution code for the `Item`, `Noodles`, `Vegetables`, and `Order` classes. The `main()` function also shows how to use those classes. There are multiple approaches to implementing this program, so your code could be slightly different.

```
open class Item(val name: String, val price: Int)

class Noodles : Item("Noodles", 10) {
    override fun toString(): String {
        return name
    }
}

class Vegetables(vararg val toppings: String) : Item("Vegetables", 5) {
    override fun toString(): String {
        if (toppings.isEmpty()) {
            return "$name Chef's Choice"
        } else {
            return name + " " + toppings.joinToString()
        }
    }
}

class Order(val orderNumber: Int) {
    private val itemList = mutableListOf<Item>()

    fun addItem(newItem: Item): Order {
        itemList.add(newItem)
        return this
    }

    fun addAll(newItems: List<Item>): Order {
        itemList.addAll(newItems)
        return this
    }

    fun print() {
        println("Order #${orderNumber}")
        var total = 0
        for (item in itemList) {
            println("${item}: ${item.price}")
            total += item.price
        }
        println("Total: ${total}")
    }
}

fun main() {
    val ordersList = mutableListOf<Order>()

    // Add an item to an order
    val order1 = Order(1)
    order1.addItem(Noodles())
    ordersList.add(order1)

    // Add multiple items individually
    val order2 = Order(2)
```

```

order2.addItem(Noodles())
order2.addItem(Vegetables())
ordersList.add(order2)

// Add a list of items at one time
val order3 = Order(3)
val items = listOf(Noodles(), Vegetables("Carrots", "Beans", "Celery"))
order3.addAll(items)
ordersList.add(order3)

// Use builder pattern
val order4 = Order(4)
    .addItem(Noodles())
    .addItem(Vegetables("Cabbage", "Onion"))
ordersList.add(order4)

// Create and add order directly
ordersList.add(
    Order(5)
        .addItem(Noodles())
        .addItem(Noodles())
        .addItem(Vegetables("Spinach"))
)

// Print out each order
for (order in ordersList) {
    order.print()
    println()
}
}

```

8. Summary

Kotlin provides functionality to help you manage and manipulate collections of data more easily through the Kotlin Standard Library. A collection can be defined as a number of objects of the same data type. There are different basic collection types in Kotlin: lists, sets, and maps. This codelab focused specifically on lists, and you'll learn more about sets and maps in future codelabs.

- A list is an ordered collection of elements of a specific type, such as a list of `Strings`.
- The index is the integer position that reflects the position of the element (e.g. `myList[2]`).
- In a list, the first element is at index 0 (e.g. `myList[0]`), and the last element is at `myList.size-1` (e.g. `myList[myList.size-1]` or `myList.last()`).
- There are two types of lists: `List` and `MutableList`.
- A `List` is read-only and cannot be modified once it has been initialized. However, you can apply operations such as `sorted()` and `reversed()` which return a new list without changing the original.
- A `MutableList` can be modified after creation such as adding, removing, or modifying elements.
- You can add a list of items to a mutable list using `addAll()`.

- Use a `while` loop to execute a block of code until the expression evaluates to false and you exit the loop.

```
while (expression) {  
  
    // While the expression is true, execute this code block  
  
}
```

- Use a `for` loop to iterate over all items of a list:

```
for (item in myList) {  
  
    // Execute this code block for each element of the list  
  
}
```

- The `vararg` modifier allows you to pass in a variable number of arguments to a function or constructor.

9. Learn more

- [Kotlin Collections Overview](#)
- [List](#)
- [List Specific Operations](#)
- [List - Learn Kotlin by Example](#)
- [Loop - Learn Kotlin by Example](#)
- [listOf\(\)](#)
- [mutableListOf\(\)](#)
- [while loops](#)
- [for loops](#)
- [vararg variable number of arguments](#)
- [this keyword](#)
- [Augmented assignments \(such as +=\)](#)